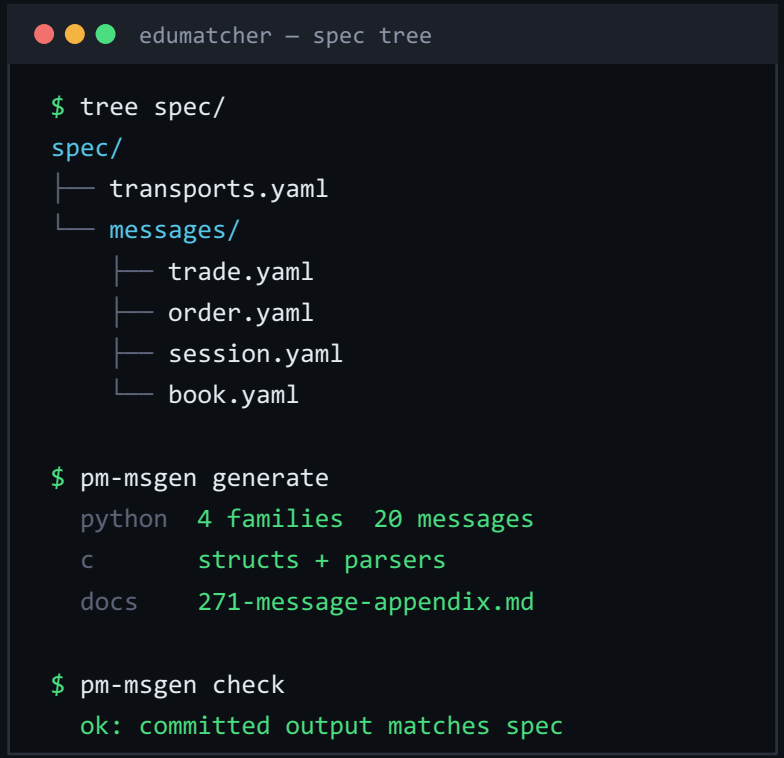
EDUMATCHER · MESSAGE LAYER

One spec. Every binding. No drift.

A beginner's guide to pm-msgen — the message generator behind the EduMatcher trading system's buses and protocols.

Phase 5.1a · trade · order · session · book



J. Persson 2026

How to read this deck

No prior knowledge of EduMatcher's internals is assumed. Every term is defined the first time it appears.

PART 1–4

The idea

Why the generator exists, what an IDL is, why we built our own, and the principles behind ours.

PART 5–8

The practice

Generating code, using it from Python and C, generating docs, and gating every build on it.

APPENDIX

The reference

Field keys, validation rules, presence regimes, binary layout rules and error codes to look up later.

One sentence to hold on to

A message generator turns a single human-readable description of a message into the code that builds it, the code that reads it, and the documentation that explains it — so those three can never disagree again.

What we will cover

01 Why a message generator at all?

02 What is an IDL?

03 Why our own, and not an off-the-shelf one?

04 The core principles of our IDL

05 Generating code, and using it in Python and C

06 Generating the message documentation

07 The class of bugs this removes

08 Validating every build

+ technical reference appendix

THE PROBLEM

01

Why do you need a message generator?

EduMatcher's components never call each other directly. They speak by sending messages. So the description of a message is the real interface of the system.

First, what is a message here?

EduMatcher is many small programs that never share memory — only messages.

A message is two frames on the wire

```
[ b'trade.executed' ,  b'{"id":"42","symbol":"ACME","price":101.5, ...}' ]
```

Frame 1 is the topic — what subscribers filter on. Frame 2 is the payload — the fields.

The same message, described three times

None of the three is authoritative. Nothing checks them against each other.

SURFACE 1

The publisher

make_* factory in models/message.py, or an inline dict in the producer

SURFACE 2

The subscriber

a topic string literal, repeated across gateways and tools

SURFACE 3

The documentation

270-message-reference.md, hand-maintained prose

No link between them

The failure this creates

Rename a field on the publisher side and the subscriber keeps compiling, keeps running, and simply stops seeing the value. No error. Just wrong.

A rename, in slow motion

Nothing in this sequence fails loudly. That is the whole problem.

Mon 09:00	A developer renames qty to quantity in the publisher.
Mon 09:01	Tests pass. The publisher is internally consistent.
Mon 09:05	Merged. CI is green — nothing compares the two sides.
Mon 11:20	The subscriber still reads payload.get("qty") → None.
Tue 14:00	Statistics show volume quietly collapsing to zero.
Thu	Two days of tracing to find a six-character rename.

The fix is not more discipline.

It is to make one file the source of truth, generate the rest from it, and fail the build when they disagree.

How big is the problem? It was counted.

A point-in-time snapshot of the EduMatcher tree. The shape matters more than the exact numbers.

92
make_* publisher factories in models/message.py

7
of those with a typed payload contract

66
topics documented by hand, across 2 222 lines

0
automated checks linking any of them

108

distinct topic string literals outside message.py, spread across 25 files

Every one is a place where a publisher-side rename goes unnoticed.

Only 7 of 92 messages declare what their fields mean. The other 85 build a dict literal inline — a reader must infer from the code whether price is display money or engine ticks.

Documentation drifts in both directions

Hand-maintained reference pages rot in two independent ways.

DRIFT A — CODE AHEAD OF DOCS

Eight topics exist only in code

- auction.indicative.{}
- book.depth.{}
- index.constituent_change_ack.{}
- index.corp_action_ack.{}
- index.history.{}
- risk.symbol_halt_ack.{} — and two more

DRIFT B — DOCS AHEAD OF CODE

Examples silently go stale

- The trades CLI example lost a column when aggressor_side was added
- book.{SYMBOL} gained tick_decimals and three surfaces needed a manual edit
- The C clients were never going to get either

Each of these needed a human to remember. That is not a system property — it is luck.

And C clients had no message types at all

The C side saw a generic bag of strings — never a trade.

calf_parser.h — before

```
typedef struct { char key[64];
                 char value[512];
} calf_field_t;

typedef struct {
  char msg_type[32];
  int  field_count;
  calf_field_t fields[64];
} calf_message_t;
```

What that costs

- Every C client re-types field names as string literals
- Every C client re-implements parsing, per field
- A rename in Python cannot reach them at all
- No types: a price and a quantity are both just text

The generator's promise

One description of a trade produces the Python class, the C struct, and the reference page — in one step, checked by CI.

The shape of the answer

One file is authoritative. Everything else is output.

SOURCE OF TRUTH

`spec/messages/trade.yaml`

human-written, reviewed

pm-msgen generate

Python binding

dataclass, constructor, parser, topic constants

C binding

typed struct, parser, validator, enum names

Documentation

Generated files are committed to the repository

A reader browsing the code never has to run the generator — and a CI check proves the committed copy still matches the spec.

VOCABULARY

02

What is an IDL?

Interface Definition Language: a small, strict language whose only job is to describe the shape of data that crosses a boundary between programs.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

An IDL, in one picture

You write the description once. A generator writes the code for each language.

Not a new programming language

An IDL has no loops, no functions and no behaviour. It only states what fields exist, what type each is, and what counts as valid.

An analogy you already know

IN THE KITCHEN

A recipe card

Ingredients and quantities, written once. Every cook in every kitchen works from the same card instead of remembering their own version.

IN A BUILDING

An architectural plan

The plumber and the electrician never speak, yet their work fits — because both read the same drawing rather than each other's walls.

IN OUR SYSTEM

A message spec

The engine, the gateways, the C clients and the docs all derive from one YAML file rather than from copies of each other.

The property all three share

There is exactly one place to change something, and a mechanical way to check that everything downstream followed. Remove either half and you are back to hoping.

What an IDL file actually contains

Four kinds of statement — nothing else.

`identity`	which family this is, and what version of it
`structure`	the messages, their fields, and each field's type
`constraints`	what makes a value legal: ranges, lengths, patterns, enumerated values
`transport`	where the message travels and what it looks like on each wire

spec/messages/heartbeat.yaml

```yaml
family: heartbeat
version: 1
messages:
  - name: engine_alive
    topic: "system.heartbeat"
    transport: [engine_pub]
    doc:
      motivation: "Liveness ping so a
        subscriber can tell a quiet
        engine from a dead one."
    fields:
      - name: sent_at
        type: float
        required: true
```

That is enough to generate a dataclass, a validating constructor, a parser and a topic constant.

Reading that file line by line

Every key earns its place. Nothing here is decoration.

`family`	The namespace. Must equal the filename stem: heartbeat.yaml.
`version`	Lets the spec evolve deliberately rather than by accident.
`name`	The message name. Becomes the class name and every function suffix.
`topic`	The string subscribers filter on. Becomes a generated constant.
`transport`	Which wires this message travels on, by name from the registry.
`doc.motivation`	Prose for the generated documentation. Never reaches the wire.
`fields`	The payload: one entry per field, each with type, rules and a unit.

> The loader is strict on purpose: an unknown key raises an error. It is not ignored, and it is not warned about.

What an IDL is deliberately not

Knowing the boundary is what keeps the tool small enough to trust.

IT DOES NOT

Own behaviour

- No business logic — matching, risk and clearing stay hand-written
- No stateful normalisation — caches and delta suppression stay in the gateways
- No opinion on how a client renders a value
- No RPC, no services, no schemas for anything but messages

IT DOES

Own message shape

- What fields exist, and what each one means
- What types they are, in every target language
- What counts as a valid value, enforced identically in Python and C
- How the same event projects onto each wire format

Drawing the line at "the generator owns the field map, the gateway owns the state" is what keeps this a message-shape tool rather than a second gateway.

BUILD VS BUY

03

Why our own IDL, and not an existing one?

Off-the-shelf IDLs exist and are excellent. Each one, applied here, would have cost something the system cannot give up.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

The candidates, and why each was rejected

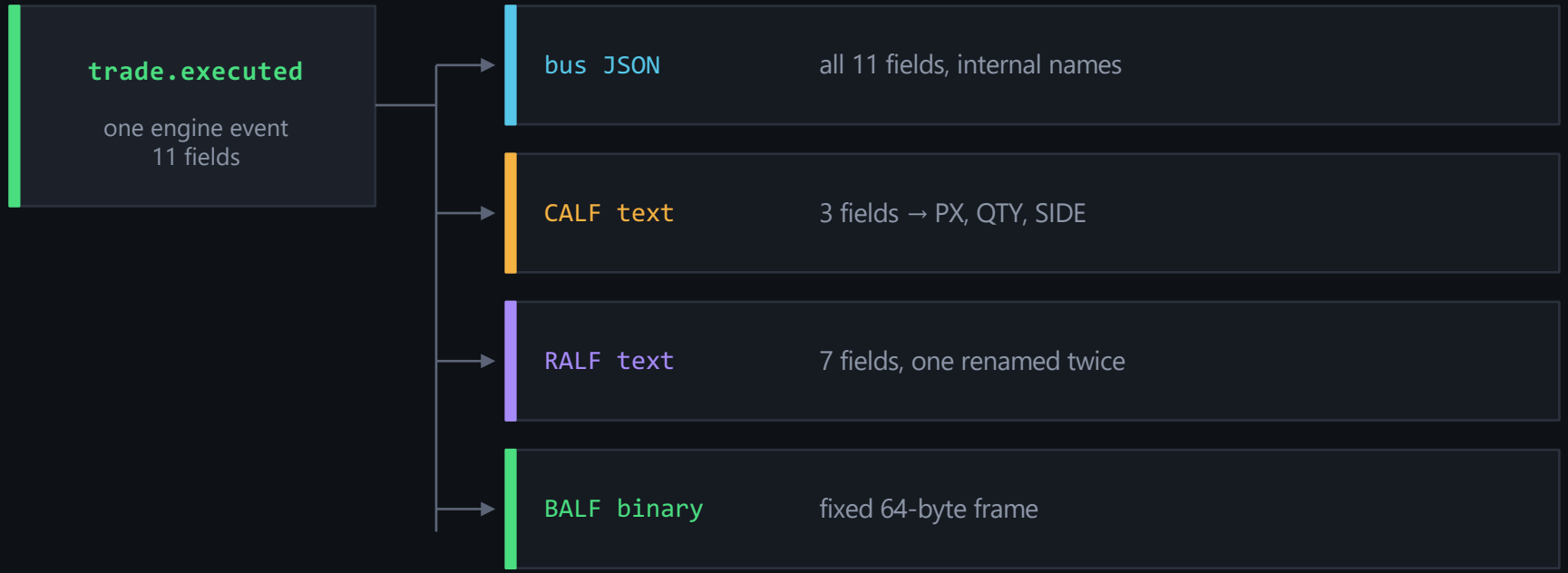
This was a deliberate decision, not an oversight.

`Protocol Buffers`	Would replace the wire format itself
`FlatBuffers`	Same problem — and EduMatcher's formats are the teaching material
`JSON Schema`	Validates JSON only. No C generation, no binary layout, no topic model
`AsyncAPI`	Closest fit and the topic model is right — but its targets are web-oriented and it cannot express BALF's fixed binary header or CALF's positional text
`Hand-written`	The status quo. Section 1 measured the result: 108 literals, 25 files

EduMatcher's CALF text and BALF binary formats are pedagogical artefacts — students read and parse them by hand. Replacing them removes the teaching value.

The real differentiator: projections

One logical event travels on four wires — and it is a different shape on each.



A projection is a subset of fields, each renamed, some supplied by the gateway rather than the message. No off-the-shelf tool models projection across heterogeneous wire formats.

The same trade, three ways

Not the same fields under three names — genuinely different content per feed.

FIELD (bus)	BUS JSON	CALF	RALF
id	id	— not public	EXEC_ID, MATCH_ID
symbol	symbol	SYM (gateway)	SYM (gateway)
buy_order_id	buy_order_id	—	BUY_ORDER_ID
price	price	PX	PX
quantity	quantity	QTY	QTY
aggressor_side	aggressor_side	SIDE	SIDE
timestamp	timestamp	TS (gateway)	TS (gateway)
tick_decimals	tick_decimals	—	—

The public CALF feed carries three of the eleven fields. A C client sees a three-field struct — not the bus payload.

So: own it, but keep it small

The cost of ownership is only acceptable because the scope is tight.

COST OF BENDING A BIG TOOL

What we avoided

- Rewriting the wire formats students learn from
- A generator plugin for a format the tool was never designed to emit
- Version pinning and upstream breakage in a teaching repo
- Explaining someone else's model before explaining ours

COST OF OWNING A SMALL ONE

What we accepted

- A YAML loader with strict, tested rejection rules
- Two emitters — Python and C — plus a docs emitter
- Determinism as a hard requirement
- A specification we must document ourselves

The specification needed is small enough that owning it is cheaper than bending one that does not fit.

PRINCIPLES

04

The core principles of our IDL

Seven ideas explain almost every rule in the language. Each one exists because a specific failure was possible without it.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

The seven principles at a glance

01

One source of truth

The spec file is authoritative; everything else is generated and committed.

02

Strict by default

An unknown key raises. Ambiguity is a spec error, never a runtime guess.

03

Presence is explicit

Optional means one of four precise things, and you must say which.

04

Units are mandatory

Every number declares what it measures — ticks are not money.

05

Coercion ≠ validation

Reading data and asserting it is correct are separate jobs.

06

Structure over convention

Records make illegal states unrepresentable instead of merely invalid.

07

Deterministic output

Byte-identical generation is what makes the CI check possible at all.

08

In short

State it once, state it precisely, and let the machine keep everything else in step.

Principle 1 — one source of truth

The spec is the only place a field is described. Everything downstream is output.

Why generated code is committed

A reader browsing the repository never needs to run the generator, and reviewers see the real diff of a spec change.

Why hand-edits cannot survive

check re-renders every artefact and diffs it. A hand-edit to a generated file fails the build exactly like a stale one.

```
$ vim spec/messages/trade.yaml   →   $ make msgen   →   $ git add spec/ src/.../generated/
```

Principle 2 — the loader is strict on purpose

Every rejection below is a spec-file error, not a wrong answer in shipped code.

REJECTED

A record inside a record

Keeps both generators non-recursive. Nothing here needs deeper structure.

REJECTED

A nullable list

An empty list already says "nothing". null would be a second spelling every reader must handle.

REJECTED

A type nothing references

It would generate a class that nothing ever constructs.

REJECTED

min_items above max_items

No list could satisfy it, so every message would fail validation at runtime.

REJECTED

A literal address in the transport registry

Symbolic config keys only — that is what keeps ports out of generated code.

REJECTED

An unknown key, anywhere

Not ignored, not warned about. A typo must not silently do nothing.

Failing early in a YAML file is cheap. Failing late on a live wire is not.

Principle 3 — presence: four regimes, stated explicitly

An optional field is one of four things on the wire. The loader will not guess which.

1	always present, "" when omitted	`required: false, default: ""`
2	always present, null when unset	`required: false, nullable: true`
3	absent entirely when unset	`nullable: true, omit_when_none: true`
4	absent when the string is empty	`required: false, omit_when_empty: true`

Regime 4 is strings only — on a number, omitting on empty would silently drop a legitimate zero. It cannot be combined with omit_when_none or with default; both are loader errors.

Why omission is modelled at all

Emitting null everywhere would be simpler grammar and a worse wire.

python

```
>>> _topic, payload = decode(
...     make_cancelled_msg("GW1", "O1"))
>>> payload
{'order_id': 'O1'}
# no client_tag key at all
```

The reasoning, from the codebase

"An ordinary single order carries none of these, and its events should not grow four empty fields to say so."

`models/message.py`

A rule only fires on a value that exists

Validation applies only when a nullable field is set. None means "not set" — not "set to something invalid" — so max_len on an absent client_tag does not fire.

27 hand-written builders already dropped keys with `if x:`. The IDL names that behaviour instead of leaving it as a habit.

Principle 4 — every number declares its unit

unit never converts anything. It makes a mismatch reviewable.

The bug that made this mandatory

trade.executed.price is display money. trade_log.price is ticks. Both are floats, so no type system catches the mismatch — it cost a full session to untangle.

trade.yaml

```yaml
- name: price
  type: float
  unit: display_price   # not ticks
  validate: { gt: 0 }
```

The legal units

`display_price` `ticks` `shares`
`epoch_seconds` `epoch_nanos`
`percent` `dimensionless` `money`

Mandatory on every numeric field. A reviewer can see it; the generator can print it in the docs.

Principle 5 — coercion and validation are different jobs

Reading a value and asserting it is correct must be separable.

FUNCTION	COERCES?	VALIDATES?	USE IT WHEN
`from_dict(payload)`	yes	no	reading history: replay, archive, audit
`validate()`	no	yes	you want to know, separately
`make_*(**kw)`	yes	yes	publishing — always
`make_*_unchecked(**kw)`	yes	no	a measured hot path only
`parse_*(frames)`	yes	yes	receiving a live message
`to_dict()`	no	no	handing back a plain dict

> Why: the spec should state the honest contract, but the system has an archive written before that contract existed. Those two needs conflict unless reading and asserting are separated.

The worked case: aggressor_side

One lenient fallback for reading, without weakening the contract.

trade.yaml

```yaml
- name: aggressor_side
  type: enum
  values: [BUY, SELL, AUCTION]
  required: true
  parse_default: ""  # NOT legal
```

python

```python
archived = {...}  # no aggressor_side
from_dict(archived).aggressor_side
  ""   — reads fine
from_dict(archived).validate()
  MessageValidationError
```

- Nothing that reads history breaks
- Every published message is still checked
- "" never becomes part of the contract
- The "" population becomes countable

Had "" been declared a legal enum value, it would have had to become a C enum member — exporting an accident into a second language and freezing it in a wire format.

Principle 6 — records: declare once, embed anywhere

A family declares record types at the top of its file and references them by name.

spec/messages/book.yaml

```yaml
types:
  BookLevel:
    doc: "One aggregated level."
    fields:
      - { name: price, type: float,
          unit: display_price }
      - { name: qty,   type: int,
          unit: shares }
messages:
  - name: book_snapshot
    fields:
      - { name: bids, type: list,
          ref: BookLevel }
```

What you get

```
>>> snap.bids[0].price
95.0        # a real BookLevel
>>> snap.validate()   # walks both ladders
```

nested = both-or-neither

A pair of keys that must travel together is a record that was flattened. A nullable record makes the half-set state unrepresentable rather than merely invalid — which is why the IDL has no co_present constraint.

Rules declared on a record's fields apply everywhere it is embedded. That is the point of declaring it once.

Principle 7 — determinism is not optional

The build gate only works if generation is byte-identical every time.

```
Same spec  ·  run twice  ·  on any machine  →  byte-identical output
```

CONSEQUENCE

Formatting is emitted, not applied

The emitter reproduces black's rules directly. Running black at generation time would make output depend on the installed version.

CONSEQUENCE

Enums become named aliases

One alias per enum keeps every use site short, so no line-wrapping rule can ever apply — however many values a future enum grows.

CONSEQUENCE

Patterns are constants, not text

A regex is interpolated by object, because a spec pattern may contain quotes or braces that would break the f-string carrying it.

A generator that occasionally reorders its own output turns the CI check into flaky failures — which is worse than not having a check at all.

Topics, transports and the registry

Two more pieces of the language, both about where a message goes.

parameterised topic

```
- name: order_ack
  topic: "order.ack.{gateway_id}"
  transport: [engine_pub]
  fields:
    - { name: gateway_id, type: string }
```

spec/transports.yaml

```
engine_pub:
  pattern: PUB
  subscriber_pattern: SUB
  address_config_key:
    ENGINE_PUB_ADDR  # symbolic
```

A topic segment can name a field

This is what removes hand-typed topic literals from every subscriber in the tree.

Never a literal address

address_config_key must be symbolic. lint rejects anything that looks like tcp://...

A topic parameter is dropped from the payload by default

order.ack.{gateway_id} names the gateway in its topic and does not repeat it in the body — matching what the hand-written builder already did.

Binary messages: BALF declares a byte layout

Instead of wire keys, a fixed frame — and every byte must be accounted for.

order.yaml — encoding: balf

```yaml
encoding:
  balf:
    msg_type: 0x20
    frame_size: 64  # 8 hdr + 56 body
    price_scale: 100000000
    layout:
      - { field: order_id,  repr: u64,
          offset: 8 }
      - { field: fill_price, repr: i64,
          offset: 16, scale: price_scale }
      - { field: symbol, repr: "char[8]",
          offset: 40 }
      - { field: side, repr: u8, offset: 48,
          enum_map: { BUY: 1, SELL: 2 } }
```

The 64-byte frame

header 0–7	generator-owned
order_id 8–15	u64
fill_price 16–23	i64 ÷ 1e8
… 24–39	declared fields
symbol 40–47	char[8]
side 48	u8
reserved 50–55	explicit gap

repr: "char[8]" is quoted — unquoted, YAML reads [as a flow sequence.

The rules the binary loader enforces

Each one turns a runtime mystery into a spec-file error.

Every body byte is covered exactly once	A gap must be an explicit reserved run, so a hole is a decision rather than an oversight
Offsets may not overlap or overrun	Two fields sharing a byte is undecodable
Every field needs a layout entry	A field with nowhere to go would silently never be sent
char[N] must equal the field's max_len	Otherwise a legal value would not fit the frame the spec describes
A binary enum needs a complete enum_map	A missing name is a value that cannot be encoded — discovered at runtime otherwise
msg_type is unique across all families	It is all a receiver has to tell frames apart
A repr must be able to carry the type	u64 cannot hold a string

The coverage rule is the one that earns its keep — a layout eight bytes short of its frame_size fails to load, loudly.

GENERATION

05

Generating the code, and using it

Four commands, one edit loop, and a small predictable set of symbols per family — in both Python and C.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

The four commands

Everything the tool does, from the command line.

`$ pm-msgen lint`	validate the spec only — no files written
`$ pm-msgen generate`	write the generated Python, C and docs
`$ pm-msgen check`	fail if committed output differs from the spec
`$ pm-msgen grep-literals`	report which topics are still hard-coded

Through the Makefile

`make msgen` `make msgen-check`

Exit codes

`0` success `1` drift — regenerate `2` the spec is broken

The normal edit loop

Three steps. Forgetting the middle one is what CI now catches.

bash

```
$ vim spec/messages/trade.yaml     # add a field, change a constraint
$ make msgen                       # regenerate
$ git add spec/ src/edumatcher/models/generated/
```

what happens if you forget

```
$ make check
- Checking generated messages match the spec (pm-msgen)...
pm-msgen check: generated output is out of date with the spec.
--- generated/trade.py (committed)
+++ generated/trade.py (from spec)
-        if self.tick_decimals > 8:
+        if self.tick_decimals > 6:
```

What one family gives you in Python

A fixed, predictable set of symbols. You never hand-write any of them.

`TradeExecuted`	frozen dataclass, one attribute per field
`TOPIC_TRADE_EXECUTED`	the topic constant — replaces every string literal
`is_trade_executed(topic)`	topic test, for messages with no parameters
`topic_*() / PREFIX_* / match_*()`	build, subscribe to and destructure a parameterised topic
`make_trade_executed(**kw)`	coerce, validate, return the two bus frames
`make_trade_executed_unchecked(**kw)`	identical frames, no validation — measured hot paths only
`parse_trade_executed(frames)`	frames to a validated object
`.from_dict() / .to_dict() / .validate()`	interop with dict-based code, plus the strictness gate
`describe_trade_executed()`	field metadata at runtime, for spy and pretty-print tools
`FAMILY, FAMILY_VERSION, FAMILY_TOPICS`	registry information, for routers

Import them from edumatcher.models.generated.trade.

Using it — publishing

make_* coerces, validates, and returns exactly two frames.

publisher.py

```
from edumatcher.models.generated.trade import
    make_trade_executed

frames = make_trade_executed(
    id="42", symbol="ACME",
    buy_order_id="b-1", sell_order_id="s-1",
    price=101.5, quantity=300,
    aggressor_side="BUY",
    timestamp=1_700_000_000.0,
    tick_decimals=2,
)
# [b'trade.executed',
#  b'{"id":"42","symbol":"ACME",...}']
publisher.send_multipart(frames)
```

Exactly two frames

The per-topic sequence number is a third frame appended by the publisher at send time — never by make_*. Adding it here would double-stamp every message.

Coercion on the way in

price=100 (an int) puts 100.0 on the wire. A missing required field raises KeyError.

It now validates

A zero price, or a payload with no aggressor_side, used to go out silently. It now raises.

Using it — consuming

parse_* coerces and validates, so a malformed payload fails at the boundary.

subscriber.py

```
from ...generated.trade import (
    is_trade_executed, parse_trade_executed)
from edumatcher.models.message import decode

frames = subscriber.recv_multipart()
topic, _payload = decode(frames)
if is_trade_executed(topic):
    trade = parse_trade_executed(frames)
    print(f"{trade.symbol} {trade.quantity}"
          f" @ {trade.price}")
```

Fail at the boundary

A malformed message raises here rather than producing a plausible-looking object that fails somewhere much deeper.

Sequence-safe

parse_* reads only the first two frames, so a sequence-stamped message parses unchanged.

A recorder is the exception

pm-stats adopted the topic constant but not the parser: a recorder records what it received. Refusing to store a message because it fails the current spec destroys the evidence you need to find out why it was malformed.

Using it — subscribing without a topic literal

For a parameterised topic the generator emits three helpers.

gateway.py

```
from ...generated.orders import (
    PREFIX_ORDER_ACK, match_order_ack,
    topic_order_ack)

sock.setsockopt_string(zmq.SUBSCRIBE,
                       PREFIX_ORDER_ACK)  # "order.ack."
gateway_id = match_order_ack(topic)  # "GW1" or None
sock.send_multipart(
    encode(topic_order_ack("GW1"), payload))
```

Why [^.]+ and not .+

match_* matches a single dot-delimited segment. A greedy .+ would make order.ack.GW1.extra match and return a subtly wrong gateway id instead of a clean None.

Migration is mechanical

`"trade.executed"` → `TOPIC_TRADE_EXECUTED`

A family is migrated when its literal count reaches zero — and a test keeps it there. trade and order are at zero today.

Using it — the hot path

make_* validates on every call. The engine publishes per match, so there is a twin.

engine/main.py::_publish_trade

```
from ...generated.trade import
    make_trade_executed_unchecked

self.pub_sock.send_multipart(
    make_trade_executed_unchecked(
        id=trade.id, symbol=trade.symbol,
        price=from_ticks(trade.price, ...),
        quantity=trade.quantity, ...))
```

Only on a measured path

It exists for the engine's trade publication loop, not for convenience. Everywhere else the validation is the point — never reach for it to silence a real finding.

CONSTRUCTION	µs/CALL	vs HAND-WRITTEN	IDENTICAL FRAMES?
the hand-written dict literal	0.96	—	n/a
generated literal, no coercion	1.12	+0.16	No
generated literal, inline coercion — ships	**1.47**	**+0.50**	**Yes**
via from_dict → dataclass → to_dict	4.03	+3.08	Yes

200 000 iterations, orjson. Dropping coercion saves 0.34 µs and would silently diverge from make_* — so it is paid.

Using it — the generated C struct

A C struct mirrors what its transport carries, not the internal bus payload.

edumatcher_trade.h

```
typedef enum {
  EDU_TRADE_EXECUTED_AGGRESSOR_SIDE_BUY = 1,
  EDU_..._SELL = 2, EDU_..._AUCTION = 3
} edu_trade_executed_aggressor_side_t;

typedef struct {
  double  price;      /* PX,  display_price */
  int64_t quantity;   /* QTY, shares */
  edu_..._side_t aggressor_side; /* SIDE */
} edu_trade_executed_calf_t;

int edu_trade_executed_calf_parse(...);
int edu_trade_executed_calf_validate(...);
```

Three fields, not eleven

C clients speak CALF and never see the bus. The struct is the projection.

No allocation, ever

Fixed-size buffers and int returns, matching the hand-written examples. A string becomes char name[max_len + 1] — which is why max_len is mandatory.

The types say what the values are, not what carries them: a scaled i64 price becomes a double; an enum byte becomes a real enum.

Using it — reading a trade in C

Two steps, mirroring from_dict and validate() in Python.

your_client.c

```
#include "edumatcher_trade.h"

calf_message_t msg;
if (calf_parse_line(line, &msg) != 0) return;

edu_trade_executed_calf_t trade;  char err[128];
int rc = edu_trade_executed_calf_parse(&msg, &trade);
if (rc != EDU_MSG_OK) {
    fprintf(stderr, "TRADE: %s\n",
            edu_msg_strerror(rc)); return; }

if (edu_trade_executed_calf_validate(
        &trade, err, sizeof(err)) != EDU_MSG_OK) {
    fprintf(stderr, "rejected: %s\n", err); return; }
```

Step 1 — parse

Coerces, and reports a missing or unparseable field.

Step 2 — validate

Enforces the declared rules. A client that prefers to show a questionable print simply skips this call.

```
CFLAGS := -std=c11 -Wall -Wextra -I. -I$(GEN_DIR)      # -I. is needed too: the generated header includes "calf_parser.h"
```

Using it — the binary path

BALF frames are fixed-size, and length is checked before anything else.

python

```python
frame = serialise_execution_report_balf(
    payload, seq_no=session.next_seq())
sock.sendall(frame)  # exactly 64 bytes

report = parse_execution_report_balf(frame)
report.fill_price  # 150.0, scaling undone
report.side        # "BUY"
```

c

```c
edu_execution_report_balf_t er;
int rc = edu_execution_report_balf_parse(
            frame, len, &er);
er.fill_price;  /* double, ÷1e8 done */
er.order_id;    /* uint64_t */
edu_..._side_to_str(er.side); /* "BUY" */
```

Why length comes first

A frame of the wrong length is not this message. Unpacking it anyway would read neighbouring bytes as field values and hand back a plausible result — so both bindings refuse rather than guess.

The 8-byte header is the generator's

Magic 0xBA, version, msg_type, flags and a u32 sequence number. It must not appear in layout, and parse_* checks all of it first.

How the two bindings are kept honest

One test compiles the committed C and feeds it lines built by the committed Python.

The property nothing could state before

Not "the generator works", but "a C client and a Python publisher read the same bytes the same way" — and reject the same values.

No new dependencies

shutil.which("cc") plus pytest.skip — no cffi, no marker, no CI install step. The proposed harness turned out to be unnecessary.

A binding that only round-trips against itself proves nothing. That lesson is section 7.

DOCUMENTATION

06

Generating the message documentation

The same spec that produces the bindings produces the reference chapter — so the docs stop being a separate thing someone has to remember.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

What the generated appendix contains

Per message, in the existing chapter's table style.

1
Motivation

The doc.motivation prose from the spec — why this message exists at all.

2
Transports

Each transport's pattern and config key, taken from the registry. Nothing restated per message.

3
Field table

Type, unit, required, constraints and description — the part that currently rots.

4
Worked examples

One per declared transport: bus JSON, a CALF or RALF line, or a BALF hexdump.

Only the transports the message actually appears on

A public-feed message shows no BALF example, and an order-entry message shows no bus example. Because the docs are generated from the same projection model as the code, they cannot describe a shape the code does not emit.

What stays hand-written, and why

The generator takes the mechanical half and leaves the narrative alone.

270-MESSAGE-REFERENCE.MD — KEPT

The narrative

- Sequence diagrams
- Subscription tables and walkthroughs
- Protocol explanations and rationale
- Everything a generator should not attempt

271-MESSAGE-APPENDIX.MD — GENERATED

The mechanical detail

- Field-by-field tables with units and constraints
- Per-transport worked examples
- Regenerated on every spec change
- Diffed by CI against the spec

The nav entry is part of the artefact

mkdocs' navigation is a hand-maintained list, not auto-discovered. A generated page that isn't in the nav builds silently and is unreachable — so generate also inserts the nav line idempotently, and check fails if the page exists without it.

Runtime metadata: docs for tools, not just readers

describe_*() hands a program the same field table the docs are built from.

spy_tool.py

```
from ...generated.trade import
    describe_trade_executed

for field in describe_trade_executed():
    unit = f" [{field['unit']}]" if ... else ""
    print(f"{field['name']:<16}"
          f"{field['type']:<8}{unit}")
```

output

```
id              string
symbol          string
price           float   [display_price]
quantity        int     [shares]
aggressor_side  enum
timestamp       float   [epoch_seconds]
tick_decimals   int     [dimensionless]
```

One description, three audiences

A human reads the appendix, a program reads describe_*(), and a compiler reads the generated struct. All three come from the same YAML lines, so none of them can be out of date on its own.

CORRECTNESS

07

The class of bugs this prevents

Not "fewer bugs" in general. One specific, mechanical failure class — and the reason a careful reviewer does not reliably catch it.

```
spec/
├── messages/
│   ├── trade.yaml
│   ├── order.yaml
│   ├── session.yaml
│   └── book.yaml
└── transports.yaml
```

The failure class, named

"No error, just wrong" — the recurring theme across this system's reviews.

A mechanical inconsistency between two surfaces that describe the same thing

WHY IT HIDES

Nothing fails

Both sides compile, both run, both pass their own tests. Each side is internally consistent.

WHY IT HIDES

The signal is data

The symptom is a missing value or a wrong number, seen far downstream and far later.

WHY IT HIDES

Review cannot scale

Catching it means holding three files in your head at once, every time anyone edits one of them.

A generator removes the class by construction: there is only one description, so there is nothing for a second one to disagree with.

Four defects this repository actually produced

Grounded in real history, not hypotheticals.

✕ **book.{SYMBOL} gained tick_decimals**

Three surfaces needed the edit. The C clients were never going to get it.

✕ **The trades CLI example lost a column**

aggressor_side was added; the example output silently went stale. Caught by a later manual audit.

✕ **Eight topics exist only in code**

Constructed by producers, absent from the hand-written reference entirely.

✕ **EodBookPayload.from_dict dropped tick_decimals**

The typed payload was not updated alongside the producer. Found only by tracing the field by hand.

Every one is a mechanical inconsistency between surfaces — exactly the class a generator removes and a reviewer does not reliably catch.

The case study: a reference implementation that lied

Writing the BALF spec meant finding an authoritative layout. Four sources disagreed.

SOURCE	EXECUTION_REPORT	order_id
910-app-balf-protocol.md — "normative"	64 bytes	u64
balf_gwy/codec.py — emits the frames	64 bytes	u64
test_balf_gwy_unit.py — asserts offset 48	64 bytes	u64
docs/examples/balf/balf_parser.{py,c}	72 bytes	char[16]

The customer reference implementation was the wrong one

Modelling order_id as a 16-byte string where the protocol defines a u64 made it exactly eight bytes too large on all six messages that carry one. A customer writing a client from it would mis-parse every single one.

And it propagated

The design document used that layout as its worked example, and even listed it under "claims that checked out" — the verification was real; the source was wrong.

Why nothing caught it

The example had a self-test. The self-test passed.

Testing against yourself

The example parser checked itself against frames the same file built. It agreed with itself perfectly — while disagreeing with the gateway that actually emits the bytes.

Testing across a boundary

The generator's tests compare against codec.py and against a different language — never against themselves. Two independent implementations must agree on the same bytes.

The rule to take away

A binding that only round-trips against itself proves nothing. Correctness claims must cross a boundary — another implementation, another language, or the real producer.

The episode is kept in the design document rather than tidied away: it is the clearest evidence that the problem is not hypothetical.

What the generator can and cannot promise

Being precise about this is what keeps the tool trustworthy.

IT CANNOT PREVENT

Bugs of meaning

- A wrong price computed by the engine
- A rule that is expressed correctly but is the wrong rule
- A message that should never have been sent
- Anything unspecified — four of roughly fifteen families have a spec today

IT DOES PREVENT

Bugs of disagreement

- A field renamed on one side only
- A type or unit that differs between languages
- Documentation describing a shape the code no longer emits
- A binary layout that no longer matches the frame it declares

It only covers what is specified

log, index and risk have no spec yet, and everything unspecified still drifts exactly as it did before. The check cannot protect a message it has never been told about.

THE BUILD GATE

08

Validating every build

Without a check, a generator is a scaffolder and the drift returns within a release. This is the part that makes the guarantee real.

What pm-msgen check actually does

It re-renders every artefact from the spec and diffs it against what is committed.

Any of these fails the build

- ✕ A spec change without regenerating
- ✕ A hand-edit to a generated file
- ✕ A missing generated file
- ✕ A generated doc page with no nav entry

The exit codes are the message

1 means "you forgot to regenerate". 2 means "your spec is wrong". A build gate needs to tell those apart — they need different fixes.

Where the gate runs

In the local build and in CI, deliberately invoked in two different ways.

make check / make pre-commit

```
make msgen-check
```

stamp-cached on spec/*.yaml and src/**/*.py — free when nothing changed

CI, code-check job

```
python -m edumatcher.msgen.cli check
```

runs alongside black, flake8 and mypy

Why CI invokes the module, not the script

The code-check job installs without the project root, so console scripts are not on PATH — only dependencies are. "Simplifying" that step would fail with command not found instead of a drift report.

A guarantee that an unrelated refactor can delete without anyone noticing is not a guarantee.

The wiring itself is tested

A test parses the Makefile and the CI config and fails if the gate disappears.

tests/test_msgen_ci_wiring.py

```
# fails if msgen-check drops out of _check
assert "msgen-check" in makefile_check_deps
# fails if the CI step disappears
assert msgen_step in ci_yaml_steps
```

Guarding the guard

The check is only useful while it runs. Asserting on the build files themselves is what keeps a well-meaning refactor from quietly removing it.

The test suite behind the generator

test_msgen_spec.py	the loader and its strictness — every rejection is a test
test_msgen_python.py	determinism, drift detection, parameterised topics, every rule
test_msgen_calf_roundtrip.py	compiled C vs Python over the CALF wire
test_msgen_balf_roundtrip.py	binary frames: byte-identity with codec.py, plus a C round-trip

Adopting a new family safely

Four steps, and the third is non-negotiable.

01	**Write spec/messages/<family>.yaml**	Run pm-msgen lint until it passes.
02	**Run pm-msgen generate**	Review the generated file as you would any other code.
03	**Write a wire-compatibility test**	Compare generated output against the existing hand-written producer. No family is adopted without one.
04	**Only then change call sites**	Adoption is incremental and reversible. There is no big-bang migration.

Two comparisons, not one — see the appendix for why byte-identity is right in one direction and wrong in the other.

Where the guarantee stands today

Four families of roughly fifteen are specified — and provably in sync.

4
families specified: trade, order, session, book

20
messages under the guarantee

6
record types declared and reused

0
topic literals left in trade and order

Phase 1–3	Generator, trade family, CI check	done
Phase 4a–4b	CALF text and BALF binary projections, Python and C	done
Phase 5.0–5.2b	Literal migration; order, session and book families	done
Phase 5.2c+	log, index and risk families	not started
Phase 6	Generated message appendix	not started

For every specified family, the committed bindings provably match the spec on main.

The whole idea, on one slide

One file describes a message. Everything else is generated, committed, and checked.

WITHOUT IT

Three descriptions

None authoritative, none checked. A rename is silent and shows up as wrong data, days later.

WITH IT

One description

Python, C and the docs derive from it. Disagreement becomes impossible by construction.

AND

A gate that proves it

CI re-renders and diffs. Forgetting to regenerate fails the build instead of merging quietly.

```
$ make msgen && make check
```

APPENDIX

09

Technical reference

Lookup material: field keys, validation rules, presence, binary layout, error codes and the helper surface.

Reference — field keys

Every key allowed on a field entry.

`name`	snake_case identifier, unique within the message
`type`	string, int, float, bool, enum, ticks, nested, list
`ref`	required for nested and list: a family-level entry under types:
`required`	default true; required: false must state its presence regime
`default`	what a producer gets when it omits the field; must be a legal value
`nullable`	the value may be None; the key is still always emitted, as null
`omit_when_none`	implies nullable, and omits the key entirely when None
`omit_when_empty`	strings only; omits the key when the value is ""
`parse_default`	what from_dict substitutes when the key is missing inbound; need not be legal
`unit`	required on every numeric field
`values`	required for type: enum; declaration order is authoritative
`validate`	the constraint block — see the next slide
`doc`	prose for the generated documentation and describe_*()

Reference — validation rules and units

validate: keys

`gt / ge`	greater than / greater or equal
`lt / le`	less than / less or equal
`max_len / min_len`	string length bounds; max_len is mandatory for any string reaching C
`min_items / max_items`	list length bounds, enforced in both bindings
`pattern`	regex; emitted as a module constant, interpolated by object

unit: values

`display_price`	money as a client sees it
`ticks`	the engine's integer price unit
`shares`	quantity
`epoch_seconds`	float seconds since epoch
`epoch_nanos`	integer nanoseconds since epoch
`percent`	a percentage value
`money`	a monetary amount that is not a price
`dimensionless`	a count or scalar with no unit

Rules fire only on set values

None means "not set", so a bound on an absent nullable field does not fire.

unit never converts

It is reviewable documentation the generator can print — not a transformation.

Reference — presence cheat sheet

Pick the regime by what the consumer must see.

REGIME	SPEC	ON THE WIRE WHEN UNSET
1 — default	required: false, default: ""	key present, value ""
2 — nullable	required: false, nullable: true	key present, value null
3 — omit on none	nullable: true, omit_when_none: true	key absent entirely
4 — omit on empty	required: false, omit_when_empty: true	key absent (strings only)

How to choose

Describe what the consumer requires. Where producers still differ, reproduce the bytes of the dominant one — producer disagreement is a real fork only when it changes what some reader sees.

Illegal combinations

omit_when_empty with omit_when_none, or with default, are loader errors: a field omits on one condition or the other, and the empty string is the absence here.

Reference — records and their limits

`types:`	family-level record declarations, emitted before the messages that embed them
`nested`	holds one record; with nullable + omit_when_none it means both-or-neither
`list`	holds many records; takes min_items and max_items
`no record in a record`	keeps both generators non-recursive
`no record on calf / balf`	a record inside a key-value line or fixed frame is an unsolved layout question
`no nullable list`	an empty list already says "nothing"
`no unreferenced type`	it would generate a class nothing constructs
`no maps`	a spec that seems to need one is describing a list of records

A message containing a record or a list gets no make_*_unchecked: that builder is a dict literal, and a record has no literal form.

Reference — C error codes

What the generated C functions return. Print them with edu_msg_strerror.

0	EDU_MSG_OK	success
-1	EDU_MSG_ERR_SHORT	frame or line too short
-2	EDU_MSG_ERR_MAGIC	bad magic byte (binary only)
-3	EDU_MSG_ERR_VERSION	unsupported version (binary only)
-4	EDU_MSG_ERR_MSGTYPE	unknown or unexpected msg_type
-5	EDU_MSG_ERR_LENGTH	length mismatch (binary only)
-6	EDU_MSG_ERR_FIELD	field missing, unparseable, or a rule failed
-7	EDU_MSG_ERR_OVERFLOW	value exceeds a fixed-size buffer

A return code is a per-function contract, not a global registry — calf_parse_line returns -1..-6 with entirely different meanings.

Reference — adoption comparisons

Byte-identity is the right assertion in one direction and the wrong one in the other.

ASSERT: EQUAL KEYS AND VALUES

Inline producer dict vs generated payload

- JSON objects are unordered
- Every consumer reads with .get
- Key order is not part of the contract
- Asserting byte-identity would be stronger than the system promises — and would block a legitimate change

ASSERT: BYTE-IDENTICAL FRAMES

Hand-written factory vs generated make_*

- Both derive from to_dict() over the same fields
- There is no excuse for a difference
- make_* and make_*_unchecked must also be byte-identical for any input
- That identity is the only thing that makes the unchecked variant safe

The trade family shows why: the producer emits tick_decimals in the middle, the typed payload emits it last. Both are correct, and nothing on the wire can tell.

Reference — glossary

Every term used in this deck.

IDL	Interface Definition Language — a small language describing data crossing a boundary
family	One YAML file of related messages, e.g. trade, order, session, book
topic	The first frame — the string subscribers filter on
payload	The second frame — the JSON body of fields
projection	A subset of a message's fields, renamed, as one transport carries it
CALF	Text key-value client feed protocol
RALF	Text post-trade protocol
BALF	Fixed-size binary frame protocol, 8-byte header plus body
coercion	Reading a value into its declared type, without judging it
validation	Asserting a value satisfies the declared rules — the only strictness gate
drift	Two descriptions of one thing that no longer agree

Where to go next

READ

The user guide page

Message Generator (pm-msgen) — commands, spec authoring, and worked examples for every binding.

READ

The design document

EduMatcher-Message-Generator.md, including the normative IDL in Appendix B.

TRY

The edit loop

Add a field to spec/messages/trade.yaml, run make msgen, and read the diff it produces.

The one habit to take away

When you need to change a message, change the spec — never the generated file, and never only one of the places that mention it. Then run make msgen and commit both. The build will tell you if you forgot.

END

Questions?

One spec. Every binding. No drift.

spec/messages/ → python · c · docs